problem. The disadvantage is that returning a pointer requires a means of managing the memory allocated to the object, and for simple types such as integers, the overhead of such memory management can exceed the cost of just returning the type by value. For any interface that uses this option, `std::shared_ptr` would be a good choice of pointer type; not only does it avoid memory leaks, because the object is destroyed once the last pointer is destroyed, but the library is in full control of the memory allocation scheme and doesn't have to use `new` and `delete`. This can be important for optimization purposes: requiring that each object in the stack be allocated separately with `new` would impose quite an overhead compared to the original non-thread-safe version.

Option 4: Provide Both Option 1 and Either Option 2 or 3

Flexibility should never be ruled out, especially in generic code. If you've chosen option 2 or 3, it's relatively easy to provide option 1 as well, and this provides users of your code the ability to choose whichever option is most appropriate for them for very little additional cost.

Example Definition of a Thread-Safe Stack

Listing 3.4 shows the class definition for a stack with no race conditions in the interface and that implements options 1 and 3: there are two overloads of `pop()`, one that takes a reference to a location in which to store the value and one that returns a `std::shared_ptr<>`. It has a simple interface, with only two functions: `push()` and `pop()`.

Listing 3.4. An outline class definition for a thread-safe stack